



University at Buffalo[®]

The State University of New York

Prepared by

Priyal Panda

Guided By

Prof. Dominic Sellitto

Summary of MGS636 Lab 4: Constructing a Local RAG Architecture

By constructing a no-code and a custom RAG architecture locally using Open WebUI and Python with Ollama as the backend LLM engine, I investigated the foundations of Retrieval-Augmented Generation (RAG) in this lab. The goal was to determine how RAG systems can incorporate context-specific data into large language models without requiring model retraining.

I started by building a simple RAG setup using Open WebUI. I compared the LLM responses with and without the knowledge collection after uploading the MGS636 course syllabus into a bespoke knowledge base. This illustrated how contextual correctness is greatly increased by include domain-specific documents. In order to see differences in output quality, I also tested by uploading several personal papers and querying them across various LLMs.

The second step was to use Python to create a RAG system from the ground up. I utilised the llama_index library to embed documents and communicate with the Ollama LLM server, and I installed important dependencies like chromadb and llama-index. I developed document ingestion, embedding creation using HuggingFace's bge-small-en, and querying using a custom prompt. I also built an ephemeral vector storage to manage semantic search. For increased speed, I later changed the configuration to use a permanent vector database, which let me reuse precomputed embeddings without constantly regenerating them.

Additionally, I wrote two scripts: ragrun.py to query existing permanent vector stores and ragcreate.py to generate new ones. I experimented with custom prompt templates using the PromptTemplate class and investigated features like similarity_top_k to improve result specificity in order to further improve the system.

My comprehension of the construction of RAG systems has improved as a result of this assignment, which also illustrated the practical use of integrating vector search with LLMs to produce highly contextual, dynamic AI applications.

Ask the following question: “What topics does this class cover?” Paste a screenshot of your prompt and response in your report.

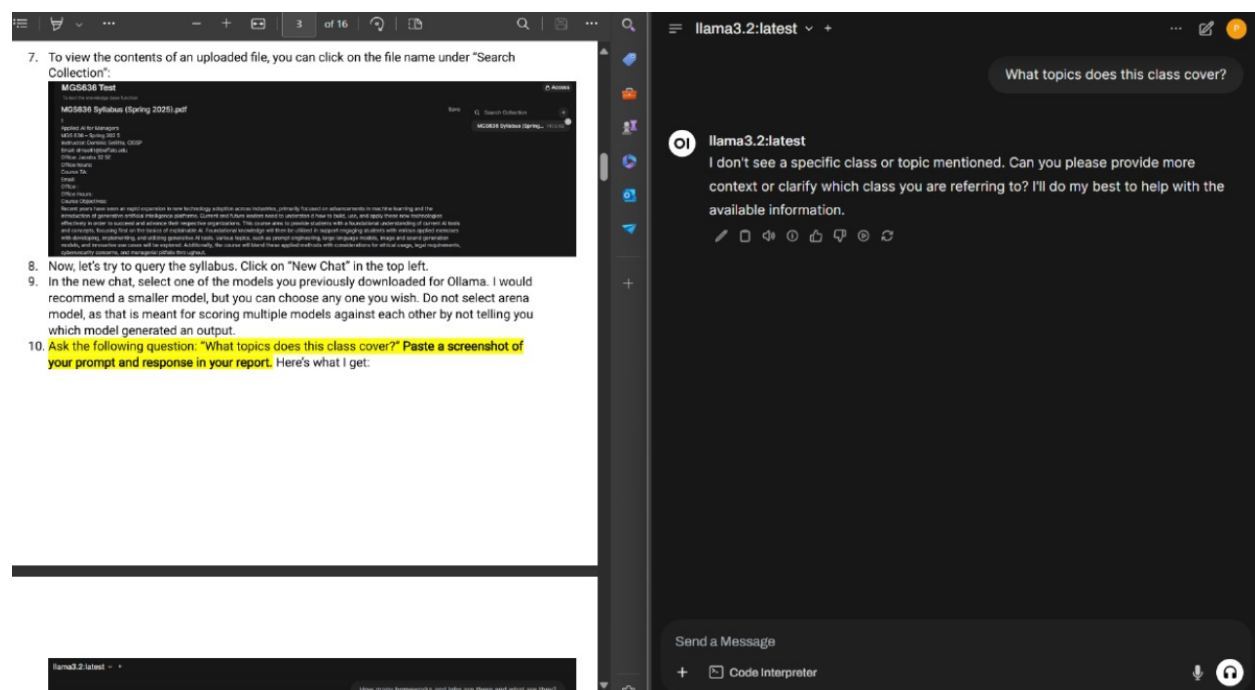


Fig1. (using llama 3.2)

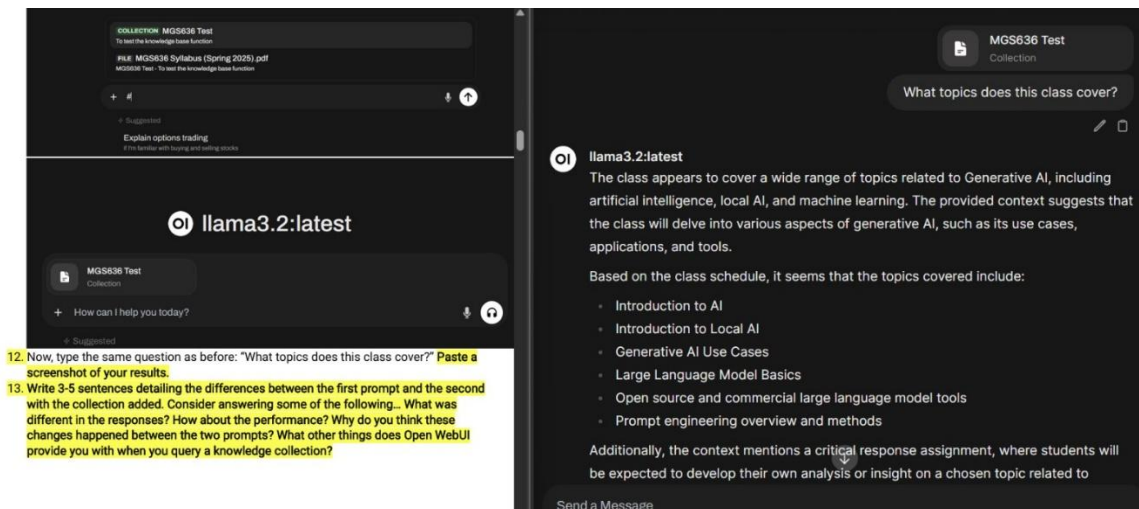


Fig2. (using llama 3.2 with pdf)

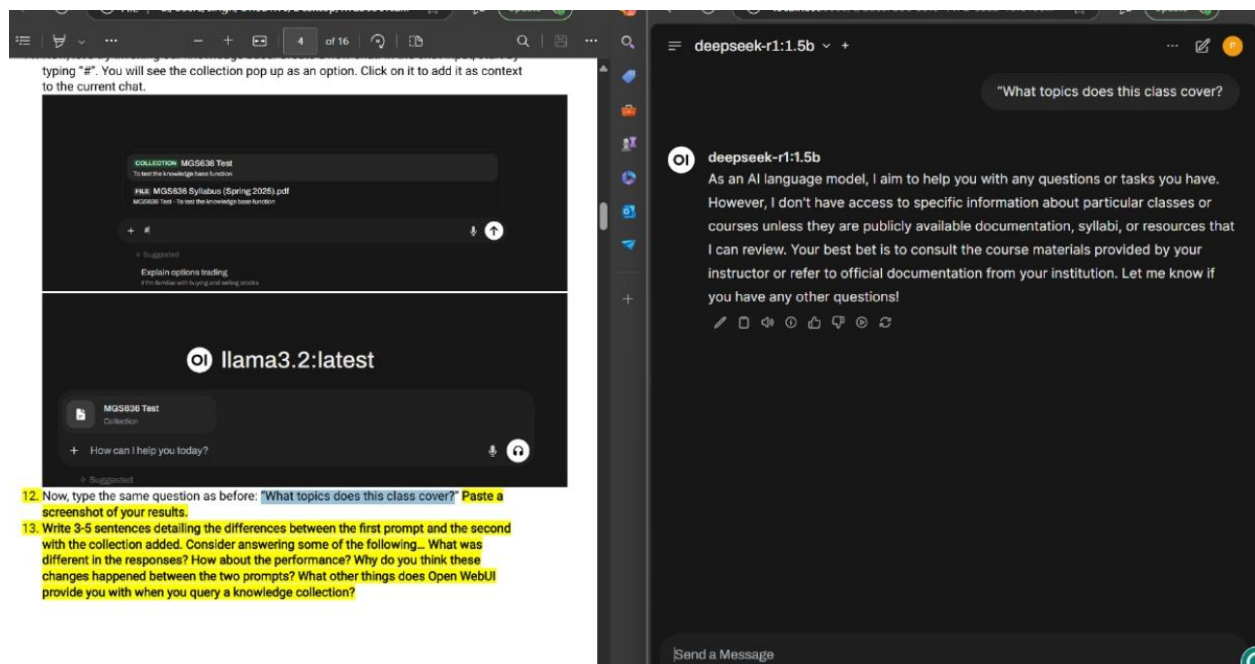


Fig3. (using deepseek without pdf)

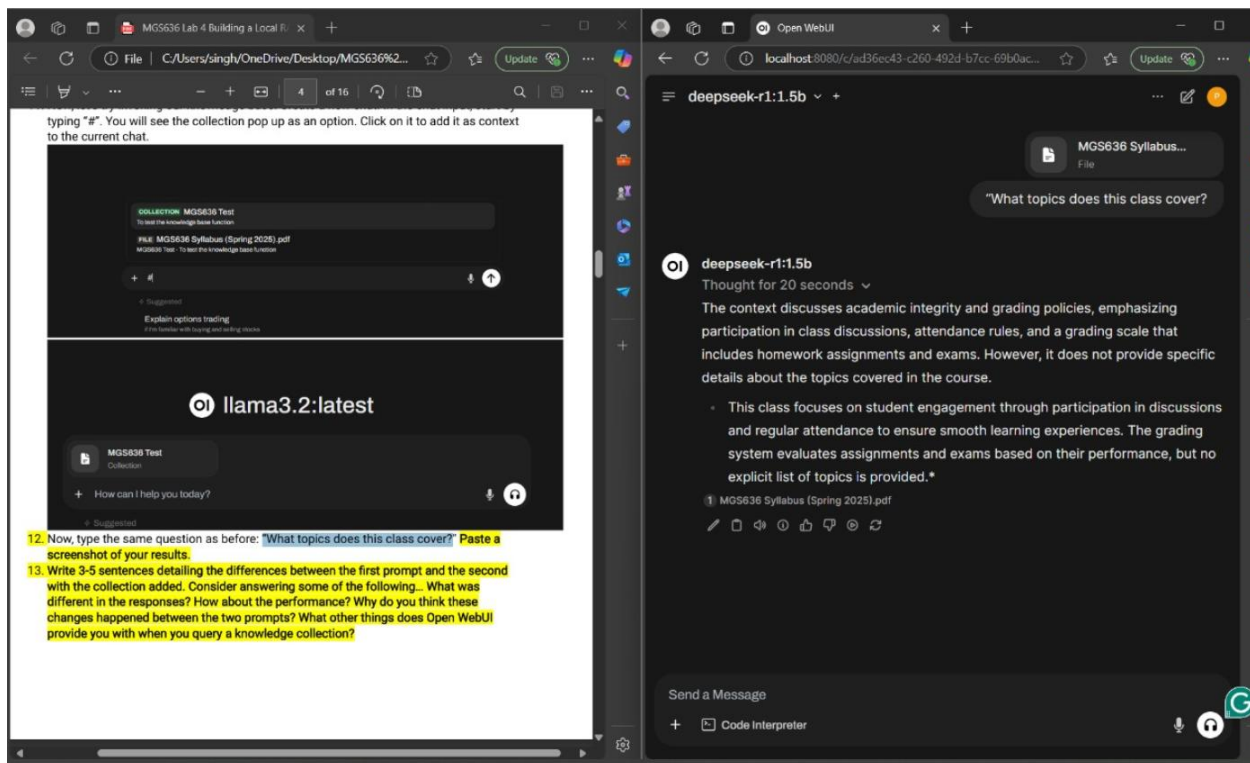
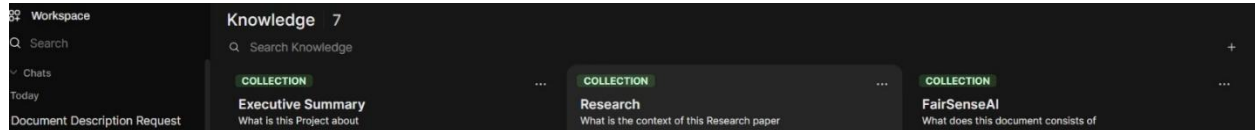


Fig1. (using deepseek with pdf)

13. Write 3-5 sentences detailing the differences between the first prompt and the second with the collection added. Consider answering some of the following... What was different in the responses? How about the performance? Why do you think these changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

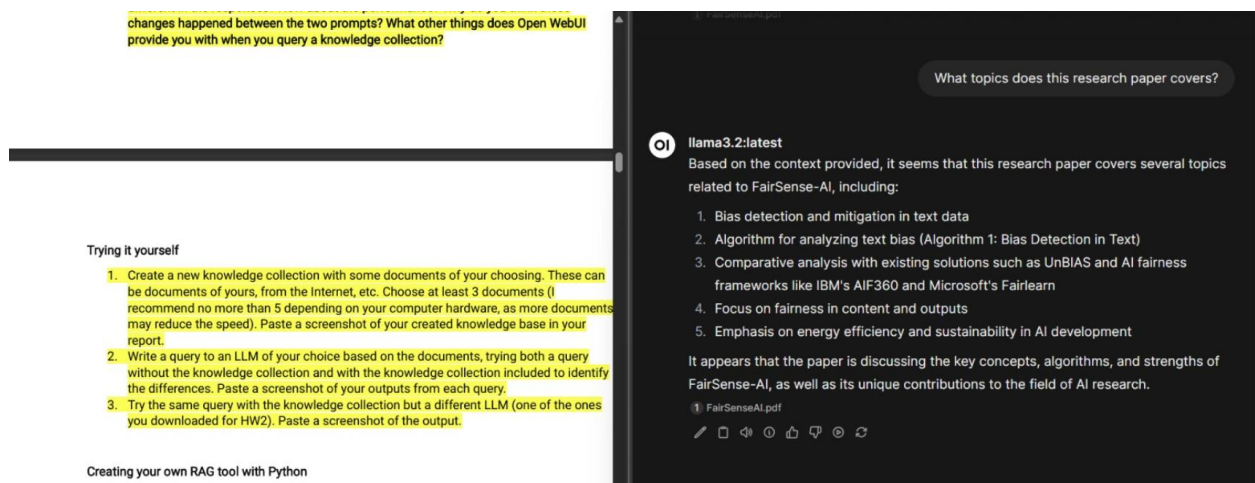
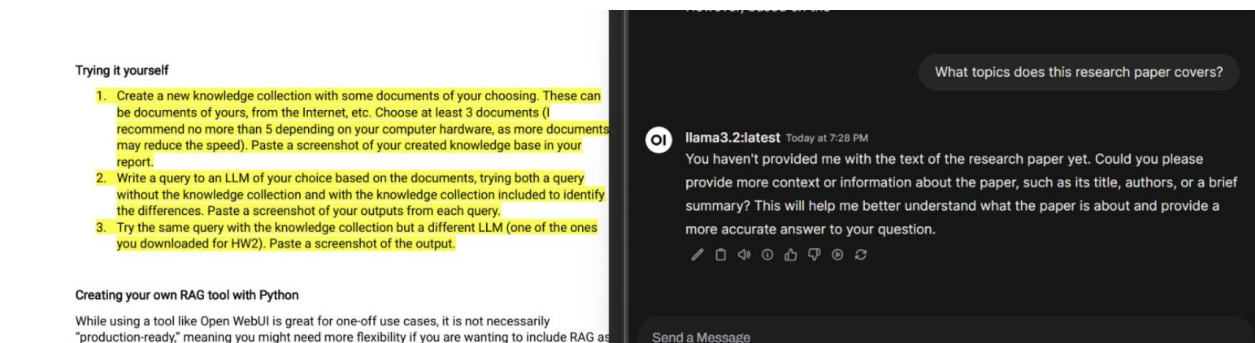
The first prompt's response, which omitted the knowledge collection (syllabus PDF), was ambiguous and requested clarification of the class to which reference was being made. But once the collection was included in the second prompt, the model offered a thorough and pertinent synopsis of the course material, including particular subjects like Prompt Engineering, Local AI, Generative AI, and more. Because the model had access to contextual data from the uploaded syllabus, it was able to provide accurate and well-informed answers, which resulted in this change in response quality. Because Open WebUI grounds the model in real documents, its use of knowledge sets greatly increases answer accuracy and relevance. In essence, it transforms the LLM into a miniature research assistant with the ability to query with document awareness.

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.



2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.

For the Fairsense Paper: (without and with knowledge base respectively)



For the Research Paper: (without and with knowledge base respectively)

changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

What topics are covered in this paper

o1 llama3.2:latest

I don't see a paper provided. Can you please share the title of the paper or provide more context about the paper, such as the field of study or the author(s)? I'll do my best to help you identify the topics covered in the paper.

changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Research Collection

What topics are covered in this paper

o1 llama3.2:latest Today at 7:33 PM

Based on the provided context, it appears that the paper covers multiple topics related to corporate governance (CG) and Environmental, Social, and Governance (ESG). Specifically, the paper explores the relationship between different CG attributes and ESG indicators. It also discusses various theoretical frameworks such as agency theory, institutional theory, resource dependency theory, and others.

Additionally, the paper provides an overview of research methodologies used in the field, including data sources and types of articles (conceptual, empirical quantitative, empirical qualitative, and mixed methods).

If you could provide more context or information about this paper, I'd be happy to try and help further.

research.pdf

For Executive Summary: (without and with knowledge base respectively)

changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

what topics are covered in this paper?

o1 llama3.2:latest

I don't see a specific paper mentioned. Can you please provide the title or a link to the paper you'd like me to analyze? I'll do my best to provide an overview of the topics covered in the paper.

Creating your own RAG tool with Python

provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Creating your own RAG tool with Python

While using a tool like Open WebUI is great for one-off use cases, it is not necessarily "production-ready," meaning you might need more flexibility if you are wanting to include RAG as part of an enterprise, consumer-facing application. In these cases, we may wish to build our

Executive Summary
Collection

what topics are covered in this paper?

Friday, April 11, 2025 7:53 PM

llama3.2:latest Today at 7:53 PM

The provided context appears to be a business plan or executive summary for a company called FraudShield AI. The main topics that seem to be covered in this document include:

1. Business Opportunity: An overview of the critical issue of digital fraud and how FraudShield AI aims to address it.
2. Product and Service: A description of the company's AI-powered fraud detection platform, including its features such as real-time detection, high accuracy, and adaptability across industries.
3. Competitive Advantages: An explanation of what sets FraudShield AI apart from other solutions, including advanced ensemble machine learning models and proprietary multi-layered detection algorithms.
4. Revenue Model: Details on the company's revenue streams, which include tiered subscription packages, custom enterprise solutions, consulting services, and advanced analytics reporting.

Send a Message

3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Creating your own RAG tool with Python

While using a tool like Open WebUI is great for one-off use cases, it is not necessarily "production-ready," meaning you might need more flexibility if you are wanting to include RAG as

Research
Collection

What topics are covered in this paper?

llava-phi3:latest

In the analysis reviewed, studies that investigate ESG performance metrics and research focusing on the link between CG attributes were found to use the Bloomberg ESG Disclosure Score (ESGDS) as a predominant dataset. The primary dataset used for investigating ESG performance is the Thomson Reuters Eikon Database ESGScore/Asset4 Score or Refinitiv score, making up 61% of the studies that explore ESG performance.

research.pdf

Send a Message

changes happened between the two prompts? What other things does Open WebUI provide you with when you query a knowledge collection?

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Creating your own RAG tool with Python

While using a tool like Open WebUI is great for one-off use cases, it is not necessarily

Executive Summary
Collection

What topics are covered in this paper?

llava-phi3:latest Today at 7:56 PM

The paper covers the following topics:

- Business Opportunity: FraudShield AI, an innovative technology startup addressing the critical and growing challenge of digital fraud across multiple industries. With global fraud losses estimated at over \$5 trillion annually, our AI-powered solution provides businesses with an advanced, adaptive fraud detection platform that offers unprecedented protection and financial security.
- Product and Service: FraudShield AI detects fraudulent activities in real-time, provides 95%+ accuracy in fraud identification, minimizes false positive rates, offers adaptive cross-industry fraud prevention, uses advanced ensemble machine learning models, proprietary multi-layered detection algorithms, rapid deployment and integration, scalable cloud-based infrastructure, transparent and explainable AI decisions.
- Revenue Model: Tiered subscription packages, Custom enterprise solutions, Consulting and implementation services. Advanced analytics reporting.

Trying it yourself

1. Create a new knowledge collection with some documents of your choosing. These can be documents of yours, from the Internet, etc. Choose at least 3 documents (I recommend no more than 5 depending on your computer hardware, as more documents may reduce the speed). Paste a screenshot of your created knowledge base in your report.
2. Write a query to an LLM of your choice based on the documents, trying both a query without the knowledge collection and with the knowledge collection included to identify the differences. Paste a screenshot of your outputs from each query.
3. Try the same query with the knowledge collection but a different LLM (one of the ones you downloaded for HW2). Paste a screenshot of the output.

Creating your own RAG tool with Python

While using a tool like Open WebUI is great for one-off use cases, it is not necessarily "production-ready," meaning you might need more flexibility if you are wanting to include RAG as part of an enterprise, consumer-facing application. In these cases, we may wish to build our own RAG architecture. In doing so, we can learn quite a bit more about how RAG works, and how to tune a RAG to meet our needs. We need three key parts in order to create a RAG from scratch: An interface, an LLM inference engine, and a vector database. We already have our inference engine (Ollama), so that really leaves us with the interface (which is going to be very simple) and our vector database. Let's start with the vector database first.

What is a vector database?

FairSenseAI
Collection

What topic does this class cover?

01

llava-phi3:latest

The class covered the topic of artificial intelligence and its impact on society, including ethical considerations, privacy concerns, and bias in algorithms. The course also included discussions on large language models and their potential applications in various fields such as education and healthcare. Additionally, the class examined the environmental impact of training these models, specifically their carbon footprints and mitigation strategies. Overall, the course aimed to provide a comprehensive understanding of AI systems and their implications for society [27], [30], [10].

FairSenseAI.pdf



Above Done with llava-phi3 using the same query

RAG Creation in Python:

In this code, we're creating a new variable (llm) to store the information about contacting our llama3.2 instance hosted by Ollama. If you want to use a different LLM, you can replace llama3.2 with the name you use to run it (you can find this by running `ollama list` like we did in homework 2. Then, we run the `llm.complete` function with a static question. This will query our llm and store the output in the response variable. Then, we just print the response! Run the code. **Paste a screenshot of the output in the command prompt.**

```
The University at Buffalo is composed of 13 colleges and schools, including:
1. College of Arts and Sciences
2. Jacobs School of Business
3. School of Education
4. School of Engineering and Applied Sciences
5. School of Medicine and Dentistry
6. Graduate School
7. Alumni Association

UB offers over 300 undergraduate majors, as well as numerous graduate and professional degree programs. The university is known for its strong programs in engineering, business, education, and health sciences.

The University at Buffalo has a diverse student body, with students from all 50 states and over 100 countries worldwide. UB's athletic teams, the Bulls, compete in the NCAA Division I Atlantic 10 Conference.

Some of the notable achievements of the University at Buffalo include:

* Ranked among the top 200 universities globally by QS World University Rankings
* Recognized as a High Research Activity university by the Carnegie Foundation
* Home to numerous research centers and institutes, including the Institute for Applied Autonomous Systems Engineering (IAASE) and the Center for Translational and Integrative Genomics (CTIG)

Overall, the University at Buffalo is a reputable institution that offers students a world-class education, cutting-edge research opportunities, and a rich cultural experience.
```

11. Replace the query with 3 other questions and paste the results as screenshots into your report.

12. Change the LLM to one of the other Ollama LLMs you downloaded during HW2. Type the same question "What is the university at buffalo?" and **paste your results.**

Question1: What is Artificial Intelligence?

```
C:\Users\priya\Downloads>py ragtest.py
Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. The term "Artificial Intelligence" was coined in 1956 by John McCarthy, a computer scientist who organized a conference on the subject at Dartmouth College.

AI systems are designed to perform tasks that typically require human intelligence, such as:

1. Learning: AI systems can learn from data and improve their performance over time.
2. Reasoning: AI systems can draw conclusions and make decisions based on the information they have been trained on.
3. Problem-solving: AI systems can solve complex problems and find creative solutions.
4. Perception: AI systems can interpret and understand data from sensors, such as images and speech.

There are several types of AI, including:

1. Narrow or Weak AI: This type of AI is designed to perform a specific task, such as facial recognition or language translation.
2. General or Strong AI: This type of AI is designed to perform any intellectual task that a human can, and has the ability to reason, learn, and apply knowledge like a human being.
3. Superintelligence: This type of AI is significantly more intelligent than the best human minds, and has the potential to solve complex problems that are currently unsolvable.

AI technologies include:

1. Machine Learning (ML): A subset of AI that involves training algorithms on data to make predictions or take actions.
2. Deep Learning (DL): A type of ML that uses neural networks with multiple layers to analyze and interpret data.
3. Natural Language Processing (NLP): A type of AI that deals with the interaction between computers and humans in natural language.
4. Robotics: The use of AI to control robots, which can perform tasks such as assembly, transportation, and customer service.

AI has many applications, including:

1. Virtual Assistants: AI-powered assistants like Siri, Alexa, and Google Assistant that can answer questions, set reminders, and perform tasks.
2. Image Recognition: AI-powered systems that can recognize objects, people, and patterns in images.
3. Self-Driving Cars: AI-powered systems that can navigate roads and make decisions about steering, braking, and acceleration.
4. Healthcare: AI-powered systems that can analyze medical data, diagnose diseases, and develop personalized treatment plans.

Overall, Artificial Intelligence has the potential to transform many aspects of our lives, from how we work and communicate to how we entertain ourselves and interact with technology.
```

Question2: What is RAG?

```
C:\Users\priya\Downloads>py ragtest.py
Retrieval-Augmented Generation (RAG) is a technique used in artificial intelligence and natural language processing (NLP). It combines the strengths of both text generation and retrieval to produce high-quality, relevant outputs.

In traditional text generation methods, models are trained on large datasets of text and generate new content based on patterns learned from the training data. However, this approach can be limited by the lack of context and relevance to the query or topic.

RAG addresses this limitation by using a retrieval step before generating text. The process involves:

1. Retrieval: A model searches through a large database of text (e.g., articles, books, or web pages) to find relevant documents that match the input query.
2. Ranking: The retrieved documents are ranked based on their relevance and importance.
3. Generation: The top-ranked document(s) are used as context to generate new text. This can be done using a variety of generation models, such as transformer-based models or sequence-to-sequence models.

RAG has several benefits over traditional generation methods:

* Improved relevance: By retrieving relevant documents, RAG ensures that the generated text is more accurate and relevant to the query.
* Contextual understanding: The retrieval step provides context for the generation step, allowing the model to better understand the topic and generate more coherent text.
* Scalability: RAG can handle large amounts of data and scale up to complex queries.

RAG has applications in various domains, including:

* Search engines: RAG can be used to improve search engine results by providing more relevant and accurate answers.
* Content generation: RAG can be used to generate high-quality content for websites, blogs, or social media platforms.
* Question answering: RAG can be used to answer questions on a specific topic by retrieving relevant documents and generating text based on that context.

In summary, Retrieval-Augmented Generation is a technique that combines retrieval and generation to produce high-quality, relevant outputs. By using a retrieval step before generating text, RAG addresses the limitations of traditional generation methods and provides improved results in various applications.
```

Question3: What is the role of Applied AI Manager?

```
C:\Users\priya\Downloads>py ragtest.py
An Applied AI Manager plays a crucial role in organizations that are leveraging Artificial Intelligence (AI) and Machine Learning (ML) to drive innovation, improve efficiency, and gain a competitive edge. The primary responsibilities of an Applied AI Manager include:

1. Developing and Implementing AI Strategies: Identifying opportunities for AI adoption, developing strategies for successful implementation, and ensuring alignment with business goals.
2. Collaborating with Cross-Functional Teams: Working closely with data scientists, product managers, engineers, and other stakeholders to integrate AI solutions into existing systems and products.
3. Data Quality and Governance: Ensuring high-quality data is used to train and validate AI models, and developing data governance policies to maintain data integrity and security.
4. Model Development and Deployment: Overseeing the development, testing, and deployment of AI models, including model selection, training, and validation.
5. AI Model Monitoring and Maintenance: Continuously monitoring AI models for performance, accuracy, and bias, and performing maintenance tasks to ensure optimal performance.
6. Staying Current with Emerging Technologies: Staying up-to-date with the latest advancements in AI and ML, including new techniques, tools, and frameworks.
7. Communicating AI Results: Effectively communicating AI-driven insights and recommendations to stakeholders, including business leaders, product managers, and customers.
8. Addressing Bias and Fairness: Identifying and mitigating bias in AI models, ensuring that they are fair, transparent, and accountable.
9. Building Trust with Stakeholders: Establishing trust with stakeholders by providing clear explanations of AI decision-making processes and results.
10. Measuring ROI and Justification: Developing metrics to measure the return on investment (ROI) of AI initiatives and justifying their use to business leaders.

To be successful, an Applied AI Manager should possess a combination of technical, business, and interpersonal skills, including:

* Strong understanding of AI, ML, and data science concepts
* Excellent communication and collaboration skills
* Business acumen and strategic thinking
* Data analysis and interpretation skills
* Ability to work in a fast-paced environment with multiple priorities
* Strong problem-solving and analytical skills

By effectively managing AI initiatives, Applied AI Managers can help organizations unlock the full potential of AI and drive business growth through innovation and efficiency.
```

12. Change the LLM to one of the other Ollama LLMs you downloaded during HW2. Type the same question "What is the university at buffalo?" and paste your results.

```
C:\Users\priya\Downloads>ollama pull orca-mini
pulling manifest
pulling 66002b78c70a... 100% 2.0 GB
pulling dd90d0f2b7ee... 100% 95 B
pulling 93ca9b3d83dc... 100% 89 B
pulling 33eb43a1488d... 100% 52 B
pulling fd52b10ee3ee... 100% 52 B
verifying sha256 digest
writing manifest
success

C:\Users\priya\Downloads>py ragtest.py
The University at Buffalo (UB) is a public research university located in Buffalo, New York. It is part of the State University of New York (SUNY) system and is one of the largest and most comprehensive universities in New York.

The university was founded in 1846 as the "Buffalo Free School" and has since grown into a major institution with over 31,000 students enrolled across its campuses. The main campus is located on the city's East Side, while other campuses are located in Amherst, Orchard Park, and Albany.

UB offers a wide range of undergraduate and graduate programs in fields such as business, engineering, health sciences, arts and sciences, and more. It is known for its research opportunities, academic excellence, and commitment to community engagement.

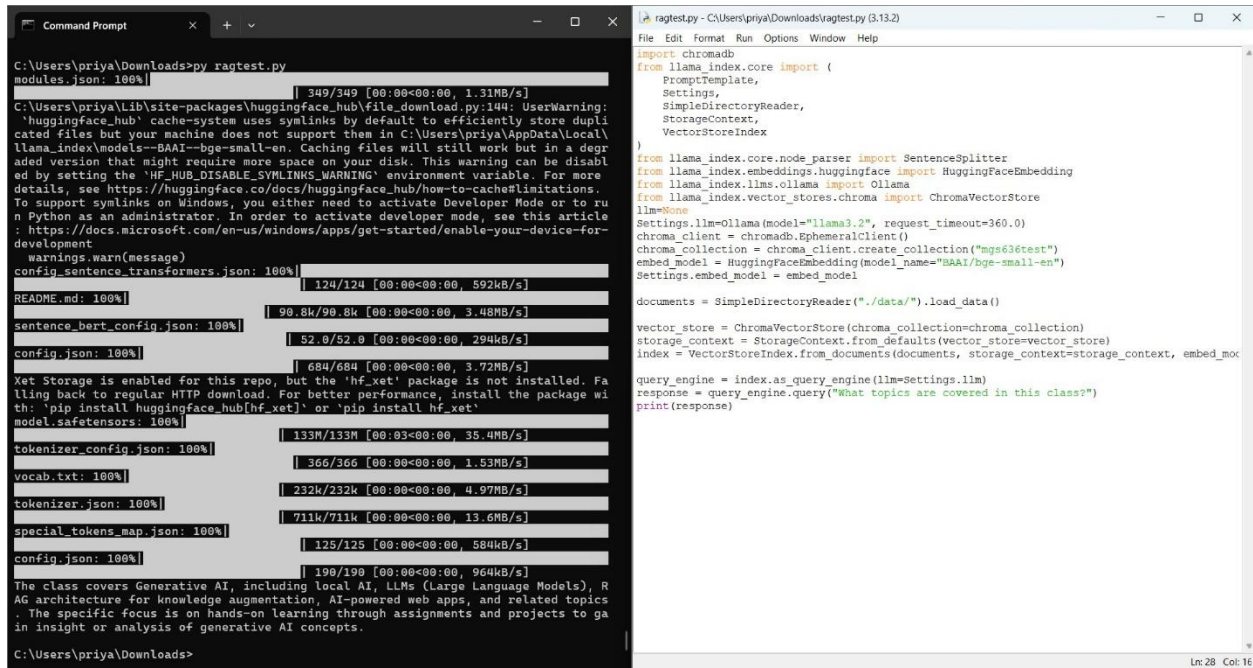
The University at Buffalo is composed of several colleges and schools, including:

1. College of Arts and Sciences
2. Jacobs School of Business
3. School of Engineering and Applied Sciences
4. School of Medicine and Health Sciences
5. School of Law

UB has a strong reputation for producing graduates who are well-prepared for careers in various fields, as well as for further education. It is also recognized for its research output and impact, with many faculty members holding prestigious awards and appointments.

Overall, the University at Buffalo is a vibrant and diverse institution that provides students with a high-quality education, innovative research opportunities, and a strong sense of community.
```


27. Run the code. It will take some time to run on your laptop, maybe even a few minutes. This is okay and expected. Once it runs, paste a screenshot of the command prompt with the results alongside your code in your IDE (like IDLE) in your report.



The screenshot shows two windows. The left window is a Command Prompt with the following output:

```
C:\Users\priya\Downloads>py ragtest.py
modules.json: 100%|██████████| 349/349 [00:00<00:00, 1.31MB/s]
C:\Users\priya\lib\site-packages\huggingface_hub\file_download.py:144: UserWarning:
'huggingface_hub' cache-system uses symlinks by default to efficiently store dupli-
cated files but your machine does not support them in C:\Users\priya\AppData\Local\
llama_index\models--BAAI--bge-small-en. Caching files will still work but in a degr-
aded version that might require more space on your disk. This warning can be disabl-
ed by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more
details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to ru-
n Python as an administrator. In order to activate developer mode, see this article
: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-
development
  warnings.warn(message)
config_sentence_transformers.json: 100%|██████████| 124/124 [00:00<00:00, 592kB/s]
README.md: 100%|██████████| 90.8k/90.8k [00:00<00:00, 3.48MB/s]
sentence_bert_config.json: 100%|██████████| 52.0/52.0 [00:00<00:00, 294kB/s]
config.json: 100%|██████████| 684/684 [00:00<00:00, 3.72MB/s]
Xet Storage is enabled for this repo, but the 'hf-xet' package is not installed. Fa-
lling back to regular HTTP download. For better performance, install the package wi-
th: 'pip install huggingface_hub[hf_xet]' or 'pip install hf_xet'
model.safetensors: 100%|██████████| 133M/133M [00:03<00:00, 35.4MB/s]
tokenizer_config.json: 100%|██████████| 366/366 [00:00<00:00, 1.53MB/s]
vocab.txt: 100%|██████████| 232k/232k [00:00<00:00, 4.97MB/s]
tokenizer.json: 100%|██████████| 711k/711k [00:00<00:00, 13.6MB/s]
special_tokens_map.json: 100%|██████████| 125/125 [00:00<00:00, 584kB/s]
config.json: 100%|██████████| 190/190 [00:00<00:00, 964kB/s]
The class covers Generative AI, including local AI, LLMs (Large Language Models), R
AG architecture for knowledge augmentation, AI-powered web apps, and related topics
. The specific focus is on hands-on learning through assignments and projects to ga-
in insight or analysis of generative AI concepts.
C:\Users\priya\Downloads>
```

The right window is a Python IDE showing the following code:

```
ragtest.py - C:\Users\priya\Downloads\ragtest.py (3.13.2)
File Edit Format Run Options Window Help

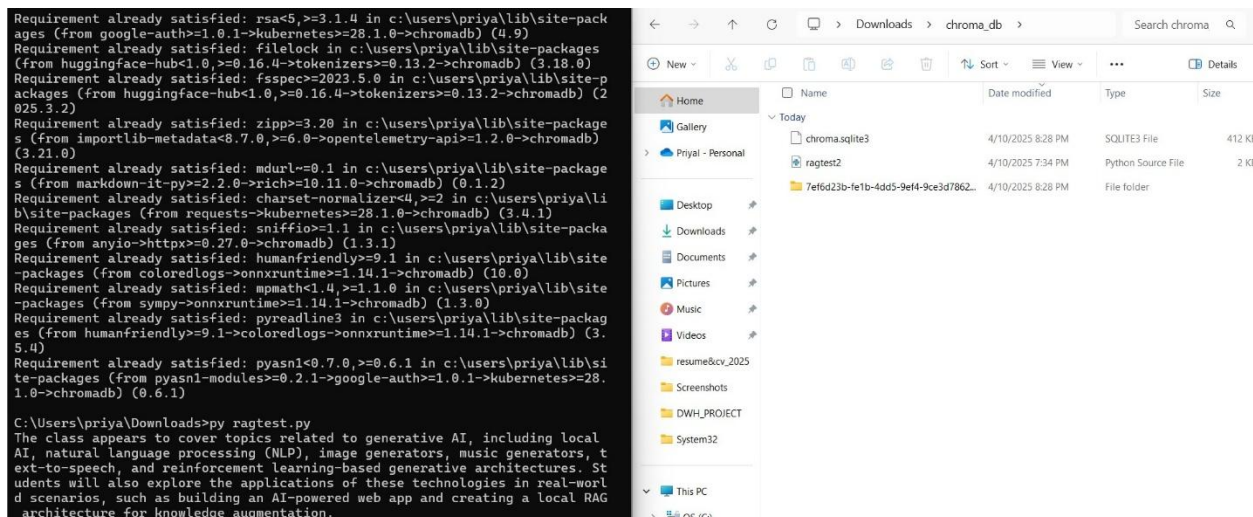
import chromadb
from llama_index.core import (
    PromptTemplate,
    Settings,
    SimpleDirectoryReader,
    StorageContext,
    VectorStoreIndex
)
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore
llm=None
Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)
chroma_client = chromadb.EphemeralClient()
chroma_collection = chroma_client.create_collection("mgs636test")
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model = embed_model

documents = SimpleDirectoryReader("./data/").load_data()

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context, embed_moc

query_engine = index.as_query_engine(llm=Settings.llm)
response = query_engine.query("What topics are covered in this class?")
print(response)
```

When complete, run the script to make sure it works. Once you are successful, paste a screenshot of your code along with a screenshot of the output from the file and another screenshot with the contents of the chroma_db folder on your computer. You should see a few files created with your chromadb database.



The screenshot shows a file explorer window with the following contents:

Name	Date modified	Type	Size
chroma.sqlite3	4/10/2025 8:28 PM	SQLITE3 File	412 KB
ragtest2	4/10/2025 7:34 PM	Python Source File	2 KB
7ef6d23b-fe1b-4d5d-9ef4-9ce3d7862...	4/10/2025 8:28 PM	File folder	

```

import chromadb
from llama_index.core import (
    PromptTemplate,
    Settings,
    SimpleDirectoryReader,
    StorageContext,
    VectorStoreIndex
)
from chromadb import PersistentClient
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore
llm=None
Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)
db = chromadb.PersistentClient(path="./chroma_db")
chroma_collection = db.get_or_create_collection("quickstart")
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model = embed_model

documents = SimpleDirectoryReader("./data/").load_data()

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
index = VectorStoreIndex.from_documents(documents, storage_context=storage_conte

query_engine = index.as_query_engine(llm=Settings.llm)
response = query_engine.query("What topics are covered in this class?")
print(response)

```

When complete, run the ragrun.py script and make sure it works. **Once you are successful, paste a screenshot of your code along with a screenshot of the response in the command prompt.**

The screenshot shows two windows side-by-side. The left window is a Windows Command Prompt with the following text:

```

Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\priya>cd downloads
C:\Users\priya\Downloads>py ragrun.py
The course appears to focus on Generative AI, with a particular emphasis on
its applications and research. Students will have the opportunity to work in
teams to develop an applied project related to Generative AI, including res
earching, writing, and presenting a paper.
C:\Users\priya\Downloads>

```

The right window is a text editor showing the code for ragrun.py:

```

ragrun.py - C:\Users\priya\Downloads\ragrun.py (3.13.3)
File Edit Format Run Options Window Help

import chromadb
from llama_index.core import (
    Settings,
    SimpleDirectoryReader,
    StorageContext,
    VectorStoreIndex
)
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore

Settings.llm = Ollama(model="llama3.2", request_timeout=360.0)
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model = embed_model

db = chromadb.PersistentClient(path="./chroma_db")
chroma_collection = db.get_or_create_collection("quickstart")

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

index = VectorStoreIndex.from_vector_store(
    vector_store=vector_store,
    storage_context=storage_context,
    embed_model=embed_model
)

query_engine = index.as_query_engine()
response = query_engine.query("What topics are covered in this class?")
print(response)

```


Building my own rag :

Basic code without any top K or PromptTemplate

```
Command Prompt
C:\Users\priya\Downloads\chroma_db>py ragrun.py
C:\Users\priya\python.exe: can't open file 'C:\\Users\\priya\\Downloads\\chr
oma_db\\ragrun.py': [Errno 2] No such file or directory

C:\Users\priya\Downloads\chroma_db>cd..

C:\Users\priya\Downloads>py ragrun.py
C:\Users\priya\python.exe: can't open file 'C:\\Users\\priya\\Downloads\\rag
run.py': [Errno 2] No such file or directory

C:\Users\priya\Downloads>python ragrun.py
python: can't open file 'C:\\Users\\priya\\Downloads\\ragrun.py': [Errno 2]
No such file or directory

C:\Users\priya\Downloads>py ragrun.py
C:\Users\priya\python.exe: can't open file 'C:\\Users\\priya\\Downloads\\rag
run.py': [Errno 2] No such file or directory

C:\Users\priya\Downloads>py ragrun.py
C:\Users\priya\python.exe: can't open file 'C:\\Users\\priya\\Downloads\\rag
run.py': [Errno 2] No such file or directory

C:\Users\priya\Downloads>cd ..

C:\Users\priya>cd documents

C:\Users\priya\Documents>py ragtest.py
The document appears to cover several topics, including AI fairness, bias de
tection, responsible AI development, and sustainability in AI. It also touch
es on AI risk management, multimodal content analysis, and intersectional bi
ases. Furthermore, the document mentions the importance of integrating fairn
ess tools into AI development pipelines and complying with regulatory requir
ements for AI fairness.
```

```
ragtest.py - C:\Users\priya\Documents\ragtest.py (3.13.2)
File Edit Format Run Options Window Help

import chromadb
from llama_index.core import (
    PromptTemplate,
    Settings,
    SimpleDirectoryReader,
    StorageContext,
    VectorStoreIndex
)
from chromadb import PersistentClient
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from llama_index.vector_stores.chroma import ChromaVectorStore
llm=None
Settings.llm=Ollama(model="llama3.2", request_timeout=360.0)
db = chromadb.PersistentClient(path="./chroma_db")
chroma_collection = db.get_or_create_collection("quickstart")
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model = embed_model

documents = SimpleDirectoryReader("./data/").load_data()

vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context, embed_moc

query_engine = index.as_query_engine(llm=Settings.llm)
response = query_engine.query("What topics are covered in this document?")
print(response)
```

Code with Top K & PromptTemplate:

```
Command Prompt
1. **FairSense-AI**: A platform that addresses AI fairness in content and ou
tputs. It provides a multi-modal content analysis tool that can analyze text
and images for subtle biases.

2. **Bias Detection and Mitigation**: The document discusses the importance
of detecting bias in AI systems and tools like FairSense-AI that help mitiga
te these biases.

3. **Fairness in Content**: The platform analyzes text and images to identif
y biases, including sexism, gender stereotypes, and visual representation.

4. **AI Risk Management**: FairSense-AI incorporates an AI risk management p
erspective, aligning with responsible AI governance frameworks.

5. **Responsible AI Development**: The document highlights the importance of
developing AI systems that are not only fair but also sustainable.

6. **Future Directions**: The authors outline several future directions for
FairSense-AI and AI fairness research, including enhancing its AI Risk Manag
ement module, increasing the range of biases it can handle, and integrating
fairness tools into AI development pipelines.

7. **Applications and Case Studies**: The document discusses various applica
tions and case studies where FairSense-AI has been used to detect biased lan
guage in workplace documents and identify representational biases in images.

8. **Sustainability and Energy Efficiency**: The platform acknowledges the i
mportance of sustainability and energy efficiency in responsible AI developm
ent, minimizing environmental harm.

9. **Intersectional Biases**: The authors mention the need to address inters
ectional biases, which involve combining attributes such as race and gender
to produce unique forms of bias.

10. **Regulatory Compliance**: FairSense-AI is expected to play a crucial ro
le in compliance with regulatory requirements for AI fairness, including the
EU AI Act and Canada AI recommendations.

Overall, the document provides an in-depth look at the topics covered by Fai
rSense-AI and its potential applications and implications for responsible AI
development.
```

```
ragtest.py - C:\Users\priya\Documents\ragtest.py (3.13.2)
File Edit Format Run Options Window Help

# Set up the embedding model
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en")
Settings.embed_model = embed_model

# Load documents
documents = SimpleDirectoryReader("./data/").load_data()

# Set up vector store and storage context
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# Create index
index = VectorStoreIndex.from_documents(
    documents,
    storage_context=storage_context,
    embed_model=embed_model
)

# Create a custom prompt template
custom_query_prompt = PromptTemplate(
    """You are an AI assistant providing helpful answers based on the provided context.

    Context information is below.
    =====
    {context_str}
    =====

    Given this information, please provide a detailed and informative answer to the question:

    If the context doesn't contain relevant information to answer the question, please state
    Focus on presenting the most important points first, and include specific details from th
    """
)

# Create query engine with custom parameters
query_engine = index.as_query_engine(
    llm=Settings.llm,
    similarity_top_k=5, # Retrieve the top 5 most similar nodes instead of the default
    text_qa_template=custom_query_prompt # Use our custom prompt template
)

# Test the query engine
response = query_engine.query("What topics are covered in this document?")
print(response)
```

I organised the project by establishing a distinct IDLE script for the primary logic and a specific file directory to house the source documents in order to construct my own RAG

(Retrieval-Augmented Generation) pipeline. I started by running the RAG setup and producing the first output by implementing the code just as it was described in the reference literature.

In order to query previously indexed documents without reprocessing them, I then changed the code to provide permanent queries by establishing a connection with a Chroma database. After the persistent query configuration was functioning properly, I added `top_k` to regulate the quantity of documents that were obtained and a custom `PromptTemplate` to adjust the results' format and tone.

You will be eligible for one bonus point if you identify the one library we imported into our code but did not use at all during this assignment, and if you do a bit of research and tell me how we may use that library (5-7 sentences).

`SentenceSplitter` from `llama_index.core.node_parser` is one library that was imported but never used in the assignment.

The purpose of this library is to divide documents into smaller sections, or "nodes," so that RAG systems can process them more efficiently. In particular, the `SentenceSplitter` divides text into sentence-level pieces, assisting in the creation of more detailed and significant vectors in the embedding database. Because the system would match specific sentences rather than complete documents, using this library would enable more accurate retrieval. Because it allows the RAG system to extract only the most pertinent sentences rather than full pages, this is very helpful when working with lengthy papers.